

1. Basics of Version Control

Version control tracks changes to files over time, enabling collaboration, history review, and rollbacks. Git is a distributed version control system, meaning each developer has a full copy of the repository.

Examples:

- Tracking changes in source code over time
- Rolling back to a previous stable version

Important:

- Every developer has their own complete copy of history in Git.

2. Installation and Configuration

Git can be installed through package managers or from git-scm.com. Configuration ensures commits are linked to your identity.

Examples:

- `git config --global user.name 'John Doe'`
- `git config --global user.email 'john@example.com'`

Important:

- You can set global or project-specific configurations.

3. Git Objects

Git stores information in objects: blob (file data), tree (directory structure), commit (snapshot), and tag (pointer to commit).

Examples:

- A blob contains the content of `README.md`
- A commit links to a tree object representing the state of the project

Important:

- Each object is identified by a hash.

4. Three Main Git Areas

Git has three areas: Working Directory (local files), Staging Area (files marked for commit), and Repository (history).

Examples:

- `git add file.txt` → stages the file
- `git commit -m 'message'` → saves staged changes

Important:

- Staging allows selective commits.

5. How Git Works

Git creates snapshots of the project rather than storing diffs. It uses cryptographic hashes (SHA-1/SHA-256) for integrity and is distributed.

Examples:

- Each commit saves the full project state
- Hashes prevent undetected changes

Important:

- Snapshots are space-efficient by reusing unchanged data.

6. Hooks

Hooks are scripts triggered by Git events (pre-commit, post-merge, etc.), often used for automation and validation.

Examples:

- pre-commit hook runs tests
- post-commit hook sends notifications

Important:

- Hooks are stored in the `.git/hooks` directory.

7. Commit and Tag

Commits are recorded changes with metadata. Tags are labels for specific commits, commonly used for release versions.

Examples:

- `git commit -m 'Fix bug in login flow'`
- `git tag v1.0.0`

Important:

- Annotated tags store more information than lightweight tags.

8. References

References are pointers to commits: branches move forward, tags stay fixed, and HEAD points to the current commit.

Examples:

- HEAD → points to current branch
- origin/main → remote branch reference

Important:

- HEAD can be detached when pointing to a commit directly.

9. Branching

Branches let you work on features or fixes independently from main development. They are lightweight and fast in Git.

Examples:

- `git branch feature-x`
- `git checkout feature-x`

Important:

- Merging brings branches together.

10. Remote Repositories

Remotes are external repositories (e.g., on GitHub, GitLab). They enable collaboration by syncing commits between machines.

Examples:

- `git remote add origin URL`
- `git fetch origin`

Important:

- 'origin' is the default name for the main remote.

11. Pushing and Pulling

Push sends local commits to a remote repository. Pull fetches and integrates remote changes into the local branch.

Examples:

- `git push origin main`
- `git pull origin main`

Important:

- Pull = fetch + merge (or rebase).

12. Semantic Versioning

Semantic versioning uses the format MAJOR.MINOR.PATCH to convey release significance.

Examples:

- 2.5.1 → major=2, minor=5, patch=1

Important:

- MAJOR for breaking changes, MINOR for features, PATCH for fixes.